Introduction to Cryptography

Lecture 17: Random and Pseudorandom Numbers

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Why Do We Need Random Numbers?

Cryptographic systems frequently require values that an attacker cannot predict. Examples include:

- Encryption keys
- Initialization vectors
- Nonces
- Session keys
- Authentication challenges
- Password salts
- Digital signature randomness

Security Requirement

If an attacker can predict a supposedly random value, the security of the entire cryptographic system may be affected.

What is a Random Number?

Informally, a random number is a number whose value cannot be predicted in advance. Consider:

$$4,\ 91,\ 16,\ 203,\ 7,\ 54, \ldots$$

A good random sequence should not reveal an obvious pattern.

For binary sequences:

$$1010011011100101 \ldots$$

we expect approximately:

$$P(0) \approx P(1) \approx \frac{1}{2}$$

Randomness in Cryptography

Randomness has two important properties:

Uniform Distribution

Possible values should occur with approximately equal probability.

Independence

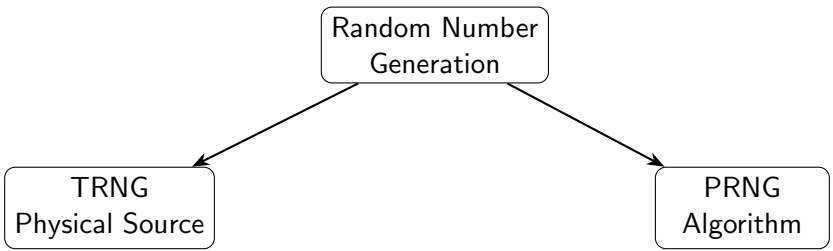
Knowing previous values should not allow an attacker to predict the next value.

These properties are especially important for cryptographic keys.

Two Ways to Generate Random Numbers

Random numbers can broadly be generated using:

1. **True Random Number Generator (TRNG)**
2. **Pseudorandom Number Generator (PRNG)**



True Random Number Generator

A **True Random Number Generator (TRNG)** uses a nondeterministic physical process.

Examples:

- Thermal noise
- Radioactive decay
- Electronic noise
- Timing variations
- Atmospheric noise

$$\boxed{\text{Physical Process} \rightarrow \text{Measurement} \rightarrow \text{Random Bits}}$$

Meaning of Nondeterministic

A deterministic process produces the same output when given the same input.
For example:

$$f(5) = 25$$

will always produce 25.

A nondeterministic physical process contains unpredictable variation.
Examples:

- Thermal noise
- Radioactive events
- Electronic noise

TRNGs attempt to measure such unpredictable behavior.

Examples of Physical Random Sources

The supplied reference identifies several physical sources:

- Pulse detectors for ionizing radiation
- Gas discharge tubes
- Leaky capacitors
- Thermal noise across electronic components
- Camera sensor noise

These natural processes provide uncertainty that can be converted into random bits.

Thermal Noise

Electronic components contain small random voltage fluctuations.
These fluctuations are called:

Thermal Noise

A hardware random-number generator may:

1. Measure the noise.
2. Amplify it.
3. Convert it into digital values.
4. Process the values.
5. Produce random bits.

TRNG using a Camera

A camera sensor can also act as a source of physical randomness.
Sensor measurements contain small unpredictable variations.
Conceptually:

$$\boxed{\text{Camera Sensor} \rightarrow \text{Sensor Noise} \rightarrow \text{Processing} \rightarrow \text{Random Bits}}$$

Historically, systems such as LavaRnd explored this approach using camera-generated noise.

What is Entropy?

In cryptography, entropy represents the uncertainty or unpredictability of a value.
Higher entropy means:

More Uncertainty

and therefore generally:

Harder to Predict

Random-number generators collect unpredictable information from:

Entropy Sources

Simple Entropy Example

Consider two systems.
System A:

1111111111

Highly predictable.

System B:

1011010010

Less predictable.

In general, a source with greater uncertainty provides more useful entropy.

Entropy Sources on Computers

RFC 4086 discusses several potential sources of randomness.
Examples include:

- Sound input
- Video input
- Disk timing
- Keyboard activity
- Mouse activity
- Interrupt timing
- Hardware noise

The objective is to collect events that are difficult for an attacker to predict.

Sound as an Entropy Source

A microphone converts analog signals into digital measurements.
Even when no intentional sound is supplied, small amounts of noise may be present.
Conceptually:

$$\text{Noise} \rightarrow \text{Microphone} \rightarrow \text{Digitizer} \rightarrow \text{Random Data}$$

Such measurements may contribute entropy to a random-number generator.

Video as an Entropy Source

Camera sensors also contain noise.
Even when a lens is covered, the sensor may produce small fluctuations.
These variations can contribute entropy.

$$\boxed{\text{Camera Noise} \rightarrow \text{Digital Values} \rightarrow \text{Randomness}}$$

The raw output generally requires additional processing.

Disk Drives as an Entropy Source

Disk operations contain small timing variations.
Possible causes include:

- Mechanical movement
- Rotational variations
- Scheduling effects
- Physical disturbances

The system may measure:

Disk Seek Timing

and extract unpredictable information.

Problem with Raw Physical Measurements

Physical measurements may be:

- Correlated
- Biased
- Noisy
- Unevenly distributed

For example:

1110111110111110

contains noticeably more 1s than 0s.

Raw physical data may therefore require processing before cryptographic use.

What is Skew?

A TRNG may produce a biased output.
For example:

$$P(1) = 0.70$$

and:

$$P(0) = 0.30$$

Instead of:

$$P(1) = P(0) = 0.50$$

This unequal distribution is referred to as:

Bias or Skew

Example of a Biased Sequence

Consider:

$$11110111101111011110$$

Count:

$$\text{Number of 1s} = 16$$

$$\text{Number of 0s} = 4$$

Therefore:

$$P(1) = \frac{16}{20} = 0.8$$

$$P(0) = \frac{4}{20} = 0.2$$

The sequence is strongly biased toward 1.

Why Bias is a Problem

Suppose an attacker knows:

$$P(1) = 0.9$$

Then the attacker can guess:

$$1$$

and be correct approximately 90% of the time.

Therefore:

$$\boxed{\text{Predictable Bias} \rightarrow \text{Reduced Randomness}}$$

This is undesirable in cryptographic systems.

Deskewing

Methods used to reduce bias are called:

$$\boxed{\text{Deskewing Techniques}}$$

The goal is to transform imperfect raw measurements into a more balanced output.

Conceptually:

$$\boxed{\text{Biased Raw Data} \rightarrow \text{Processing} \rightarrow \text{Improved Random Output}}$$

Hash Functions for Conditioning

One approach is to process entropy-source data using a cryptographic hash function. Conceptually:

$$\text{Raw Entropy} \xrightarrow{\text{Hash}} \text{Fixed-Length Output}$$

A hash function can mix information from many input bits.

Important

Modern implementations should use currently approved cryptographic hash functions rather than obsolete designs.

Combining Multiple Entropy Sources

Instead of depending on only one source, a system can combine multiple sources.
For example:

$$E_1 = \text{Keyboard Timing}$$

$$E_2 = \text{Mouse Timing}$$

$$E_3 = \text{Hardware Noise}$$

$$E_4 = \text{Interrupt Timing}$$

Then:

$$E_1||E_2||E_3||E_4 \xrightarrow{\text{Conditioning}} \text{Random Output}$$

Operating-System Randomness

Modern operating systems provide random-number generation facilities.
The general idea is:

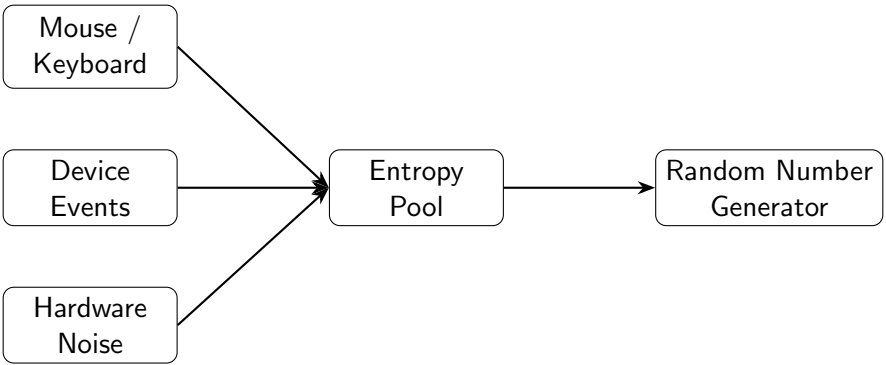
$$\boxed{\text{System Events} \rightarrow \text{Entropy Collection} \rightarrow \text{Internal State} \rightarrow \text{Random Output}}$$

Possible event sources include:

- Hardware events
- Device activity
- Interrupt timing
- Hardware random-number generators

Entropy Pool Concept

An operating system may combine entropy into an internal pool.



What is a Pseudorandom Number Generator?

A **Pseudorandom Number Generator (PRNG)** uses a deterministic algorithm.
It starts from an initial value called:

$$\boxed{\text{Seed}}$$

The seed is expanded into a much longer sequence.

$$\boxed{\text{Seed} \rightarrow \text{PRNG} \rightarrow R_1, R_2, R_3, \ldots}$$

Why is it Called Pseudorandom?

The output may appear random.
However, mathematically:

$$\boxed{\text{Same Seed} \rightarrow \text{Same Sequence}}$$

Therefore, the generator is deterministic.

Example:

$$\mathit{Seed} = 25$$

might always generate:

$$83, 17, 42, 91, \ldots$$

when used with the same algorithm.

Seed Security

The seed is extremely important.
Suppose an attacker learns the seed.
Then, for a deterministic PRNG:

$$\boxed{\text{Known Seed} + \text{Known Algorithm} \rightarrow \text{Predictable Output}}$$

Therefore, cryptographic PRNGs require unpredictable seeds and appropriate internal-state protection.

TRNG vs PRNG

Feature	**TRNG**	**PRNG**
Source	Physical process	Algorithm
Deterministic	No	Yes
Seed	Not in same sense	Required
Speed	Often slower	Usually fast
Repeatability	No	Yes with same seed
Use	Entropy generation	Large random streams

How TRNG and PRNG Work Together

In practical cryptographic systems, both ideas may be combined.

$$\boxed{\text{Physical Entropy} \rightarrow \text{Seed} \rightarrow \text{Cryptographic PRNG} \rightarrow \text{Random Values}}$$

This gives:

- Unpredictable initialization
- Efficient generation
- Large amounts of random output

Requirements of Cryptographic Randomness

A secure random-number generator should provide:

1. Unpredictability
2. Good statistical behavior
3. Appropriate entropy
4. Secure initialization
5. Resistance to state compromise where applicable

Remember

Passing statistical tests alone does not automatically prove that a generator is cryptographically secure.

Randomness vs Appearance of Randomness

Consider:

1101011001011100

It looks random.
But suppose it was generated by a completely predictable algorithm.
Then an attacker who knows the algorithm and state may predict the future output.

Therefore:

$$\boxed{\text{Looks Random} \neq \text{Cryptographically Unpredictable}}$$

Cryptographic Applications

Random values are required in many cryptographic operations.

- Symmetric-key generation
- Public-key generation
- Nonces
- Initialization vectors
- Session identifiers
- Challenge-response authentication
- Salts
- Digital signatures

Example: Random Key Generation

Suppose an encryption algorithm requires a 128-bit secret key.
Then:

$$K \in \{0,1\}^{128}$$

The number of possible keys is:

$$\boxed{2^{128}}$$

But this protection assumes that the key-generation process can select keys unpredictably from the available key space.

Example: Predictable Randomness

Imagine generating a security key using only the current hour:

$$K = f(\text{hour})$$

There are only:

$$24$$

possible values.
An attacker can simply try all possibilities.

Lesson

Large key length does not help if the randomness used to create the key is weak.

Classroom Numerical: Bias

Suppose a random source produces 1000 bits:

650 ones

and:

350 zeros

Calculate the observed probabilities.

$$P(1) = \frac{650}{1000}$$

$$P(0) = \frac{350}{1000}$$

Therefore:

$$\boxed{P(1) = 0.65}$$

Classroom Numerical: Ideal Binary Source

Suppose a perfectly balanced binary source generates:

$$10,000$$

bits.
Approximately how many 0s and 1s would we expect?
For:

$$P(0) = P(1) = 0.5$$

Expected zeros:

$$10,000 \times 0.5 = 5,000$$

Expected ones:

$$10,000 \times 0.5 = 5,000$$

Real random sequences need not contain exactly equal counts.

Classroom Activity

Classify each item as primarily associated with a TRNG or PRNG.

1. Thermal noise
2. Deterministic algorithm with a seed
3. Radioactive decay
4. Camera-sensor noise
5. Algorithmically generated random sequence

Answers:

$$1 : T, \quad 2 : P, \quad 3 : T, \quad 4 : T, \quad 5 : P$$

Common Student Confusions

Random means equal numbers of 0s and 1s?

No. Over a sufficiently large sample, frequencies should generally approach the expected distribution.

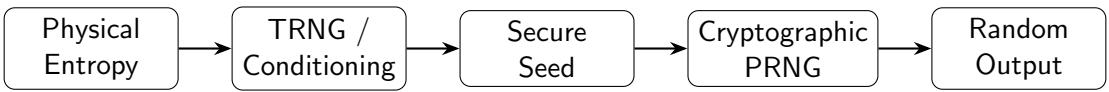
PRNG means insecure?

No. Cryptographically secure PRNGs are widely used when designed and seeded correctly.

TRNG output can be used directly?

Not always. Raw physical sources may require conditioning or other processing.

TRNG and PRNG in One Diagram



One-Slide Revision

TRNG

Physical unpredictable source

↓

Entropy

↓

Conditioning

PRNG

Seed

↓

Deterministic Algorithm

↓

Long Random-Looking Sequence

Key Points to Remember

1. TRNGs obtain randomness from nondeterministic sources.
2. Physical measurements can provide entropy.
3. Raw entropy sources can contain bias or correlation.
4. Deskewing or conditioning can improve the output.
5. PRNGs are deterministic and start from a seed.
6. The same seed produces the same PRNG sequence.
7. Cryptographic PRNGs require unpredictable initialization.
8. Secure random numbers are fundamental to cryptography.

Quick Classroom Questions

1. What is a TRNG?
2. What is a PRNG?
3. What is entropy?
4. Give three physical entropy sources.
5. What is skew?
6. Why is bias undesirable?
7. What is deskewing?
8. What is the role of a PRNG seed?
9. Why can the same seed be dangerous if reused or exposed?
10. Give four cryptographic applications requiring randomness.

Descriptive Questions

1. Explain the working principle of a True Random Number Generator with suitable entropy sources.
2. Differentiate between TRNG and PRNG with suitable examples.
3. Explain skew in a random bit stream. Why is deskewing required?
4. Explain how physical activities and system events can contribute entropy to an operating system.
5. Discuss the importance of secure random-number generation in cryptographic systems.

References

1. William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson.
2. D. Eastlake, J. Schiller, and S. Crocker, RFC 4086, *Randomness Requirements for Security*.
3. NIST, publications and recommendations concerning random-bit generation.
4. Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone, *Handbook of Applied Cryptography*.

Your Feedback Matters!

Please scan the QR code and share your feedback about today's lecture.



TCS-392 – Introduction to Cryptography
Lecture 17 – Random and Pseudorandom Numbers

Thank You

Questions?